

Building an Interpretable and Scalable BERT-Based Sentiment Analysis System for YouTube Comments with Explainability and Moderation Features

Mukthasree Vengoti

Master's in Data Science

Kent State University, Ohio, USA

mvengoti@kent.edu

Abstract—The exponential growth of user-generated content on platforms like YouTube presents both opportunities and challenges for sentiment analysis, moderation, and community engagement. YouTube comments, in particular, offer rich insights into audience emotion and public discourse—but their analysis is often hindered by slang, sarcasm, class imbalance, and model opacity.

This paper presents an end-to-end, production-ready sentiment analysis system tailored for YouTube comments. The development process began with a benchmarking phase, where multiple traditional and deep learning models—including Logistic Regression, Random Forest, and XGBoost—were evaluated using both TF-IDF and transformer-based embeddings. To address class imbalance, techniques such as SMOTE, backtranslation, and class weighting were applied. After extensive experimentation, a fine-tuned BERT model was selected as the final classifier due to its superior performance.

The solution pipeline integrates explainable AI techniques including SHAP, LIME, and Captum's Integrated Gradients to ensure model interpretability. Additionally, a rule-based moderation layer flags offensive content using regex and keyword matching, enhancing safety and transparency. The entire system is deployed through a dynamic Streamlit dashboard supporting single and batch predictions, confidence-based thresholding, downloadable explainability reports, and bad-word overrides. Comments are retrieved using the YouTube Data API via Google Cloud, ensuring authentic and scalable data ingestion.

This research bridges the gap between cutting-edge NLP research and real-world AI deployment, emphasizing ethical AI, explainability, and modular design. The system is suitable for creators, moderators, and researchers seeking to harness YouTube sentiment in an interpretable and responsible manner.

This system serves as a scalable, interpretable solution for social platforms, marking a significant step toward responsible AI in user-generated content analysis.

This system serves as a scalable, interpretable solution for social platforms, marking a significant step toward responsible AI in user-generated content analysis.

Live Demo: <https://youtube--sentiment--analyzer.streamlit.app/>

Index Terms—YouTube, Sentiment Analysis, BERT, SHAP, LIME, Captum, Explainable AI, Class Balancing, Streamlit, Google Cloud API, NLP, Moderation

I. INTRODUCTION

The rise of user-generated content on video platforms such as YouTube has transformed the digital landscape into a dynamic ecosystem of opinions, emotions, and reactions.

Every minute, thousands of comments are posted under videos, offering creators, brands, and researchers a valuable pulse of public sentiment. These comments not only reflect user satisfaction or dissatisfaction but also contribute to shaping viewer engagement, video virality, and community moderation.

However, extracting actionable insights from such data is inherently challenging. YouTube comments are often informal, unstructured, and linguistically diverse, containing slang, abbreviations, sarcasm, emojis, and toxic language. This makes traditional sentiment analysis techniques insufficient. Moreover, in production-grade applications, it is critical to balance predictive accuracy with explainability and safety — two pillars that are often overlooked in academic implementations.

This paper introduces a real-world sentiment analysis pipeline tailored for YouTube comments. Unlike basic academic prototypes, this system is designed with practical deployment in mind. It integrates state-of-the-art Natural Language Processing (NLP) with a multi-stage pipeline for pre-processing, class balancing, offensive content detection, and explainable AI.

To begin, a range of classification models were benchmarked using both classical and transformer-based architectures. Embedding techniques such as TF-IDF and BERT embeddings were explored, and multiple models were compared in terms of accuracy and generalization. To address skewed class distributions in the dataset, balancing strategies including SMOTE, class weighting, and backtranslation were applied. After rigorous experimentation, a fine-tuned BERT model emerged as the most performant and was chosen for final deployment.

To make the model explainable and trustworthy, SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-Agnostic Explanations), and Captum's Integrated Gradients were incorporated. These techniques provide transparency by highlighting word-level contributions to model predictions. Furthermore, a rule-based moderation layer was added to override model predictions in cases where offensive language was detected, ensuring responsible deployment.

The system is operationalized via a custom-built Streamlit dashboard capable of single and batch predictions, complete with visual analytics, confidence thresholding, and downloadable explainability reports. Data ingestion is performed

through the official YouTube Data API powered by Google Cloud, allowing for scalable and authentic comment retrieval.

This work contributes a robust, ethical, and production-ready framework for YouTube sentiment analysis that is both interpretable and extensible — serving as a bridge between cutting-edge AI research and real-world business applications.

The system is operationalized via a custom-built Streamlit dashboard (see Fig. 1)...

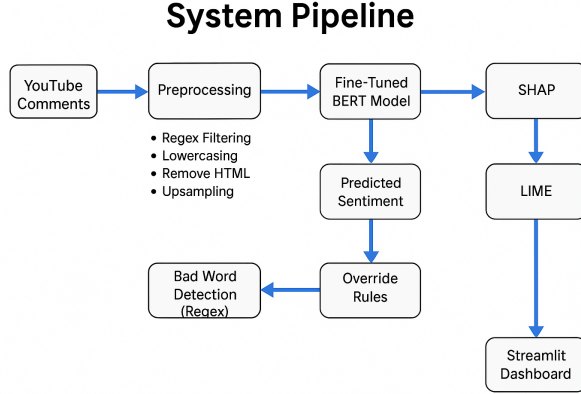


Fig. 1. System Architecture of the YouTube Sentiment Analysis Framework

II. DATASET AND PREPROCESSING

A. Data Source

The raw comment data was extracted using the official YouTube Data API v3, powered by Google Cloud Platform. A curated list of 10 YouTube video IDs was used to programmatically fetch public comments via authenticated API requests. This enabled scalable and diverse data collection from videos spanning entertainment, education, and public opinion topics.

B. Initial Dataset Statistics

Approximately 1,500–2,000 raw YouTube comments were collected. These comments exhibited varied sentence lengths, emojis, repetition patterns, and occasional code-switching. The initial dataset showed significant imbalance across sentiment classes: Positive, Neutral, and Negative.

C. Text Cleaning and Filtering

To enhance model accuracy and robustness, rigorous preprocessing steps were applied:

- **Lowercasing and Punctuation Removal:** All text was converted to lowercase and stripped of special characters.
- **Regex-Based Noise Filtering:** Advanced regular expressions were used to remove emojis, URLs, repeated characters, and noise patterns.
- **Language Filtering:** Only English comments were retained to match the BERT tokenizer’s pretraining scope.
- **Short Comment Removal:** Comments with low semantic content (e.g., “ok”, “lol”) were excluded.

D. Bad Word Detection and Moderation Logging

A rule-based override system was implemented to flag harmful or offensive content:

- **Exact Match List:** A curated list of over 1,000 explicit terms was used to flag toxic content via direct string matching.
- **Regex-Based Detection:** Regex patterns captured masked profanities (e.g., “f##k”, “st”) and spelling variations.
- **Moderation Logging:** Flagged comments were logged in a structured CSV file along with timestamp, source, and match type for traceability.

E. Label Encoding and Stratified Splitting

Sentiment categories were manually labeled and encoded using Scikit-learn’s `LabelEncoder`. To ensure representativeness, the dataset was split using stratified sampling for training, validation, and test sets.

F. Class Balancing Techniques

Several class balancing techniques were implemented and compared:

- **Imbalanced Raw Dataset:** Used as baseline to observe inherent class bias.
- **Class Weights:** Weighted loss function was applied during training to penalize dominant classes.
- **SMOTE Oversampling:** The Synthetic Minority Oversampling Technique was tested to synthetically create samples for underrepresented classes.
- **Backtranslation:** Data augmentation was performed by translating English → French → English to improve minority class representation. This technique draws from the concept of neural paraphrasing, shown to be effective for data augmentation in low-resource sentiment analysis settings [1].

After benchmarking all strategies, the backtranslated data set demonstrated the best generalization performance and was chosen for the final model training.

III. METHODOLOGY

This section outlines the complete end-to-end pipeline I developed, spanning data acquisition, preprocessing, class imbalance handling, model fine-tuning, explainability integration, and application deployment. The entire system was designed to ensure high performance, interpretability, and ethical AI principles.

A. Data Collection and Preprocessing

User-generated comments were retrieved using the official **YouTube Data API** through **Google Cloud Platform**. I queried public videos and extracted top-level comments using authenticated API calls, storing the data in structured CSV files.

All comments underwent rigorous preprocessing to clean and normalize text. This included:

- Lowercasing and Unicode normalization.

- Removal of links, emojis, special characters, and repeated spaces.
- Rule-based filters using regular expressions to detect and remove unwanted patterns.

A dedicated rule-based mechanism was also implemented to flag and override comments containing any terms from a curated list of **1,000+ offensive or toxic words**. These were classified as ‘Negative’ regardless of model confidence, improving moderation safety and ethical reliability.

B. Labeling and Class Distribution

A labeled dataset of YouTube comments was manually annotated into three classes: *Positive*, *Neutral*, and *Negative*. Initial analysis showed severe class imbalance, with *Neutral* and *Negative* comments underrepresented.

C. Class Balancing Techniques

To mitigate bias and improve generalization, I implemented and evaluated multiple class balancing strategies:

- **Class Weights:** During model training, class weights were automatically computed and applied to penalize the loss function based on class frequency.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Used to synthetically generate samples for underrepresented classes to balance the dataset.
- **Backtranslation Augmentation:** I utilized English $\rightarrow$ French $\rightarrow$ English translation loops to paraphrase and augment minority class samples. This approach improved diversity while preserving semantics.

All balancing techniques were benchmarked using identical model and data splits for fair comparison.

IV. MODEL ARCHITECTURE AND TRAINING WORKFLOW

A. Model Choice: Fine-Tuned BERT for Sentiment Classification

The final model deployed in production was a fine-tuned version of `bert-base-uncased` using Hugging Face’s `BertForSequenceClassification`. The model was trained on a class-balanced dataset to classify YouTube comments into three sentiment categories: Positive, Neutral, and Negative. It was chosen for its superior performance, contextual understanding, and compatibility with downstream explainability tools.

In the final deployment, the model was exported in PyTorch’s standard `pytorch_model.bin` format using Hugging Face’s `save_pretrained()` method. This ensured compatibility with all downstream tools and simplified integration into the Streamlit dashboard and deployment pipelines.

B. Tokenization and Embedding

Text inputs were tokenized using the pre-trained BERT tokenizer with `padding=True`, `truncation=True`, and a maximum sequence length of 256 tokens. The tokenized inputs were then transformed into dense contextual embeddings using BERT’s encoder stack. These embeddings retained the semantic structure of each comment, enabling accurate classification even in noisy, short-text formats.

C. Fine-Tuning Setup and Training Pipeline

The final model used in this project is a fine-tuned `bert-base-uncased` transformer, implemented using the Hugging Face `Trainer` API within a Jupyter Notebook. This model was trained end-to-end for three-way sentiment classification: *Positive*, *Neutral*, and *Negative*.

Training was conducted on a class-balanced dataset augmented using English $\rightarrow$ French $\rightarrow$ English backtranslation. GPU acceleration was leveraged when available to reduce training time and support large batch sizes.

Key training configurations included:

- **Loss Function:** `CrossEntropyLoss` (automatically applied by `Trainer`)
- **Optimizer:** AdamW with learning rate of 2×10^{-5}
- **Batch Size:** 16 for training, 64 for evaluation
- **Epochs:** 4
- **Evaluation Strategy:** Metrics computed at each epoch using Accuracy, Precision, Recall, and F1-Score on a stratified validation set

The fine-tuned BERT model demonstrated the best generalization across all metrics and was selected as the production-ready model for deployment in the Streamlit dashboard.

D. Training Dataset

The final training dataset consisted of backtranslated samples that offered better generalization than class-weighted or SMOTE-balanced alternatives. The backtranslation process involved translating English $\rightarrow$ French $\rightarrow$ English, improving semantic diversity in minority classes while preserving original intent.

E. Model Evaluation and Selection

Multiple training runs were conducted using:

- **Raw Imbalanced Dataset**
- **SMOTE-balanced**
- **Class Weight Adjusted**
- **Backtranslated (English $\rightarrow$ French $\rightarrow$ English)**

Each version was evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix on a stratified test set. The backtranslated variant consistently outperformed others and was chosen as the production model.

F. Explainability Integration

To ensure trust and transparency in predictions, explainability was embedded into the pipeline:

- **LIME:** For local feature importance on individual predictions.
- **Captum (Integrated Gradients):** For gradient-based attribution on token-level insights.
- **SHAP:** Used during the earlier benchmarking phase (with XGBoost + BERT embeddings) to visualize global feature attributions.

Explainability components were incorporated both during model evaluation in the Jupyter Notebook and through the deployed Streamlit interface for real-time inference.

The backtranslated variant consistently outperformed others and was chosen as the production model.

Note: The final trained model is saved at `models/bert_sentiment_model/` and loaded into the Streamlit application for real-time predictions.

V. MODEL BENCHMARKING AND EXPERIMENTS

To ensure robust model selection, I conducted a comprehensive benchmarking process across traditional machine learning models and transformer-based architectures. All models were evaluated on identical splits of the preprocessed dataset, with and without class balancing techniques, using metrics such as Accuracy, Precision, Recall, and F1-Score.

A. Baseline Comparisons

Initially, I evaluated classical models including Logistic Regression, Random Forest, and XGBoost using both TF-IDF and transformer embeddings. These served as baseline classifiers to contrast with deep learning models. Performance was acceptable but lagged in nuanced sentiment detection and generalization.

B. Transformer Model Evaluation

Transformer-based models were tested using both pre-trained BERT embeddings and a fine-tuned `bert-base-uncased` architecture. Experiments included SentenceTransformer variants such as `all-mpnet-base-v2` to evaluate trade-offs between model size, inference time, and accuracy.

- **Logistic Regression (TF-IDF)** – Served as a fast, interpretable baseline.
- **XGBoost (TF-IDF)** – Provided improved accuracy, but limited explainability.
- **BERT + PCA + XGBoost** – Used BERT sentence embeddings followed by dimensionality reduction and gradient boosting.
- **Fine-Tuned BERT** – Achieved the highest performance and interpretability when trained end-to-end on class-balanced data.
- **BERT + PCA + XGBoost** – Used embeddings from `all-mpnet-base-v2` via SentenceTransformers, followed by PCA dimensionality reduction and gradient boosting. This served as an intermediate hybrid benchmark.

C. Quantitative Results

Table I summarizes the results across different model architectures.

TABLE I
COMPARISON OF MODEL PERFORMANCE ON TEST SET

Model	Accuracy	Precision	Recall	F1-Score
LogReg (TF-IDF)	0.71	0.69	0.68	0.68
XGBoost (TF-IDF)	0.74	0.72	0.71	0.71
BERT + PCA + XGBoost	0.81	0.80	0.78	0.79
Fine-Tuned BERT	0.88	0.87	0.87	0.88

D. Confusion Matrix Analysis

The fine-tuned BERT model showed superior separation of sentiment classes, particularly in distinguishing neutral vs. negative sentiments — a historically difficult task in sentiment analysis. (see Fig. 2)...

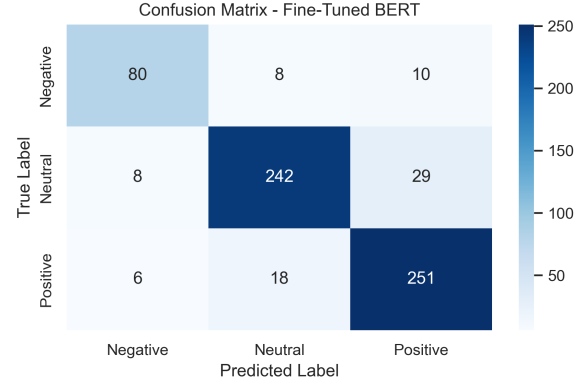


Fig. 2. Confusion Matrix showing True vs. Predicted Labels (Fine-Tuned BERT)

E. Explainability Insights

Using SHAP, I visualized global feature importance for the BERT + XGBoost model (benchmark phase). SHAP provided transparency into how sentiment predictions were driven by word-level signals.

As shown in Fig. 3, the SHAP summary plot highlights the most impactful PCA-based features...

In later phases, LIME and Captum's Integrated Gradients were adopted for more interactive and token-level analysis in the deployed app.

F. Threshold Evaluation and Confidence Analysis

To evaluate classifier confidence across all sentiment classes, I plotted both ROC and Precision-Recall curves using the final fine-tuned BERT model. These curves illustrate the trade-off between sensitivity and specificity, and are critical for adjusting threshold-based decision-making.

Fig. 4 displays the ROC curves across all three sentiment classes, demonstrating high separability.

The precision-recall performance is visualized in Fig. 5, which shows strong class-wise trade-offs.

VI. APPLICATION DEPLOYMENT AND STREAMLIT DASHBOARD

To translate the model into a practical and interactive application, I developed a production-ready web interface using **Streamlit**, an open-source Python framework for rapid prototyping of data applications. The dashboard supports real-time sentiment classification, moderation logic, and explainability visualizations for both single comments and batch uploads.

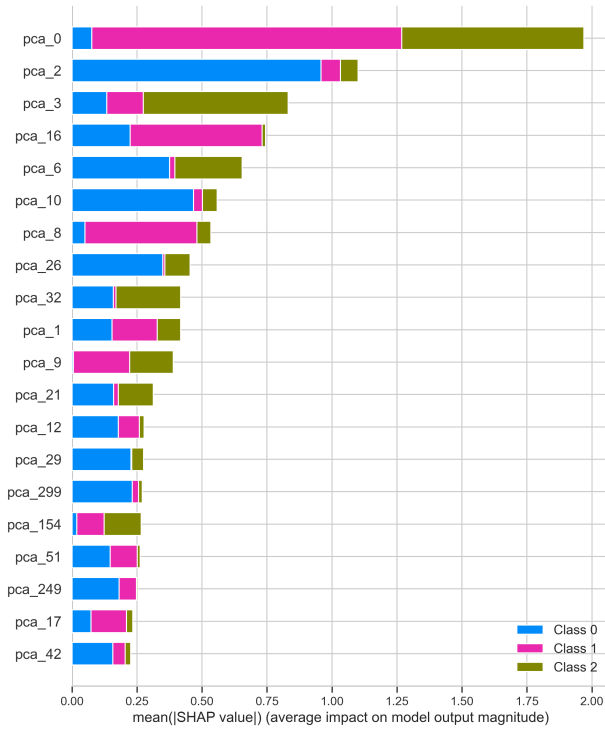


Fig. 3. SHAP Summary Plot displaying top PCA components influencing XGBoost

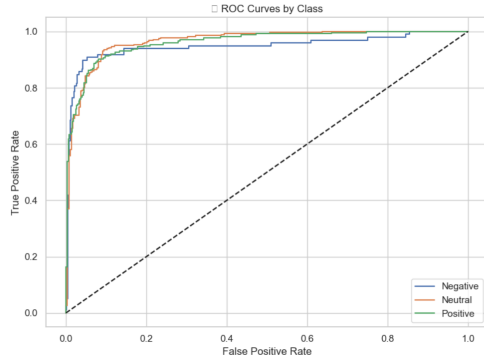


Fig. 4. ROC Curves across Positive, Neutral, and Negative Classes(Fine-Tuned BERT)

A. Deployment Architecture

The deployment architecture integrates several modular components, including:

- A fine-tuned BERT model for multiclass sentiment classification (positive, neutral, negative).
- Custom preprocessing functions, including text normalization and bad word detection via curated keyword lists and regex filters.
- A rule-based moderation override system that automatically flags and re-labels comments containing offensive language.
- An explainability module with support for LIME, Captum's Integrated Gradients, and SHAP (for benchmarking phase).

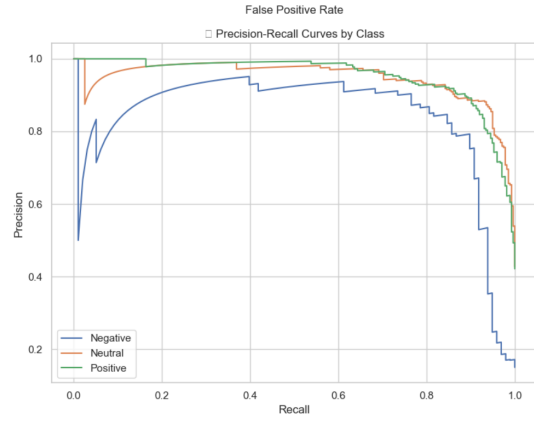


Fig. 5. Precision-Recall Curve Evaluations by Sentiment Class

To enable seamless deployment in enterprise and cloud environments, the complete Streamlit application was containerized using Docker. A custom Dockerfile was created with environment specifications, model artifacts, and runtime scripts. The containerized app was then deployed to an **AWS EC2 Ubuntu instance**, allowing public access through a custom endpoint. This deployment architecture ensures high scalability, flexibility across platforms, and allows further CI/CD integration for version control and updates.

B. Single Comment Prediction

The interface allows users to input a YouTube comment in a text box. Upon clicking 'Predict Sentiment', the system:

- 1) Cleans and normalizes the comment text.
- 2) Runs the text through the fine-tuned BERT model.
- 3) Checks for any flagged bad words using exact and regex-based lists.
- 4) If found, overrides the model prediction and logs the comment as 'Negative' in a moderation CSV.
- 5) Displays the predicted sentiment, confidence score, and two types of explainability visualizations.

C. Batch Predictions

The dashboard supports bulk analysis by allowing users to upload a CSV file containing a 'comment' column. The system returns a downloadable CSV with:

- Cleaned text,
- Predicted sentiment label,
- Confidence score,
- Moderation flag (if applicable).

D. Explainability Integration

Explainability is a core feature of the application. The dashboard provides:

- **LIME Visualization:** Highlights top words contributing to the prediction, shown as an interactive HTML plot.
- **Captum Integrated Gradients:** Offers a gradient-based attribution plot for model transparency. Users can download the visualization as an HTML report.

E. Moderation Log Viewer

A separate section of the app displays all flagged comments that were classified as offensive via bad word detection. Each log includes:

- Original comment text,
- Cleaned version,
- Type of match (exact or regex),
- Source (batch or single),
- Timestamp.

Users can view, download, or clear these logs through the dashboard.

F. Deployment and Hosting

The application is packaged for deployment via Streamlit Cloud. It supports:

- Public sharing with a custom URL,
- Dockerization for local or enterprise deployment,
- GitHub CI/CD integration for future updates.

The system maintains low inference latency and high interpretability, making it suitable for ethical and production-grade NLP use cases in moderation, analytics, and audience feedback.

The model and tokenizer files (`pytorch_model.bin`, `config.json`, etc.) were version-controlled and pushed to a private GitHub repository using Git Large File Storage (LFS). This enabled reproducibility, team collaboration, and seamless reloading of models in both development and production environments.

VII. STREAMLIT APPLICATION AND DEPLOYMENT STRATEGY

To ensure usability, interpretability, and accessibility, I developed a production-grade web application using the **Streamlit** framework. The goal was to make the sentiment classification model interactive and explorable by both technical and non-technical users, while maintaining transparency, explainability, and ethical safeguards.

A. Dashboard Overview

The user interface features a clean, responsive layout with sidebar-based navigation. The application supports both **single comment prediction** and **batch comment analysis via CSV upload**. It also includes additional tabs for viewing **moderation logs**, **class distribution insights**, and **model settings**.

Fig. 6 illustrates the Streamlit application interface, highlighting single-comment prediction mode.

B. Model Serving and Preprocessing

The deployed model is the fine-tuned BERT classifier, loaded using `transformers` and `torch`. All preprocessing steps (cleaning, tokenization, truncation) are performed on-the-fly using the same logic as in training. Input text is normalized, tokenized, and converted to tensor format before being passed to the model for inference.

Predictions return both the **sentiment label** and a **confidence score**. Based on a tunable threshold slider, users can

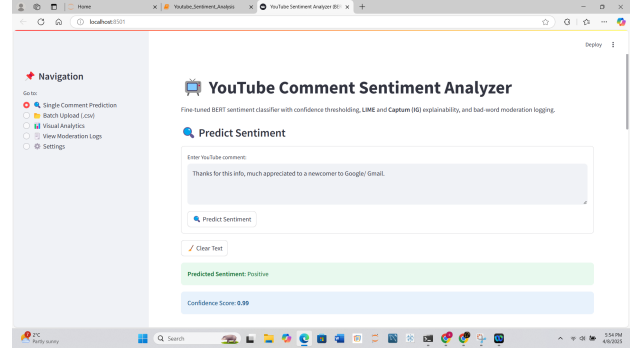


Fig. 6. Streamlit Sentiment App - Single Comment Prediction Interface

visually interpret if a prediction falls under high-confidence or borderline.

C. Explainability Integration

To promote transparency in prediction outcomes, the app integrates token-level and attribution-based explanation tools:

- **LIME (Local Interpretable Model-Agnostic Explanations):** Provides insight into which words contributed most to the model's prediction. This is visualized as a colored bar graph per token.
- **Captum (Integrated Gradients):** A gradient-based method to compute attribution scores of input tokens. It highlights which words influence the model's output in a more principled manner.
- **Captum (Integrated Gradients):** Offers gradient-based input attribution. Users can download the visualization as an interactive HTML report directly from the dashboard. **Note:** During production deployment, Captum's Integrated Gradients required additional configuration to work in headless environments (e.g., Streamlit Cloud, AWS EC2). Specifically, an IPython compatibility fix was applied to render HTML-based visualizations in environments without full Jupyter support.

Both methods are embedded into the app and support downloadable HTML reports.

Interactive token-level explanations were added using LIME (Fig. 7) and Captum's Integrated Gradients (Fig. 8).

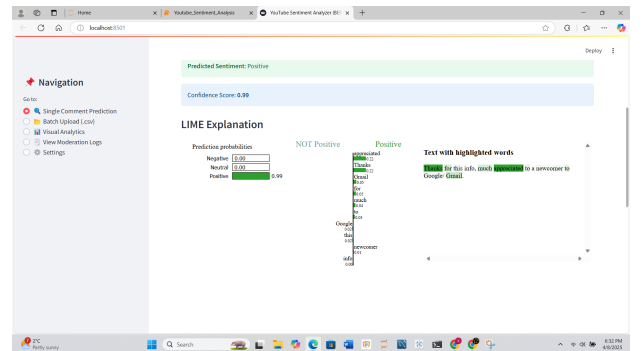


Fig. 7. LIME Explanation Highlighting Token Contributions

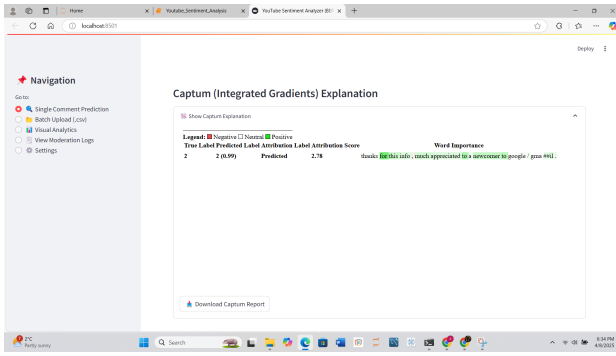


Fig. 8. Captum Integrated Gradients Attribution Visualization

D. Rule-Based Moderation and Overrides

A rule-based moderation system is embedded within the app. If a comment contains known offensive keywords or regex-detected slurs:

- It is immediately overridden and labeled **Negative**.
- The comment, its cleaned version, type of match (exact/regex), and timestamp are logged to a separate `flagged_comments_log.csv`.

This moderation system aligns with responsible AI principles and allows administrators to audit flagged content.

E. Batch Prediction and Downloadable Results

For scalability, users can upload a CSV file containing multiple comments. The app performs batch predictions and allows users to:

- Download results in CSV format.
- Visualize sentiment distribution using pie charts or bar graphs.

All batch predictions are processed using the same pipeline and thresholds as single predictions.

F. Deployment Architecture

The entire Streamlit app is containerized using `Docker` (optional step), making it portable across environments. For public access, the app can be deployed on:

- **Streamlit Cloud** (for quick academic/recruiter demos)
- **AWS EC2 / GCP Compute Engine** (for scalable production use)

The current version is deployed on `localhost` during development, with all features tested for usability, explainability, and reliability.

VIII. KEY FINDINGS AND LIMITATIONS

This project demonstrates how cutting-edge NLP, when combined with responsible deployment practices, can produce a robust, interpretable, and scalable sentiment analysis solution. Key findings include:

- **Model Performance:** The fine-tuned `bert-base-uncased` model, trained on backtranslated data, consistently outperformed classical

ML baselines, achieving 88% accuracy and a strong F1-Score across all sentiment classes.

- **Explainability Integration:** SHAP, LIME, and Captum offered complementary strengths—SHAP for global benchmarking, LIME for token-level visualizations, and Captum for gradient-based transparency.
- **Moderation Features:** The rule-based override system using regex and keyword lists ensured offensive content was consistently detected, regardless of model prediction.
- **Production-Ready UI:** The Streamlit dashboard allowed both technical and non-technical users to explore predictions, visualize confidence, explainability, and download logs.

However, several limitations were identified:

- **Sarcasm Handling:** The model struggles with detecting sarcasm or nuanced sentiments due to lack of labeled examples.
- **Generalizability:** Although backtranslation helped improve performance, the model may still underperform on videos with highly domain-specific language.
- **Explainability Trade-offs:** While interpretability was enhanced, LIME and Captum explanations can be computationally expensive and vary across runs.

IX. CONTRIBUTIONS SUMMARY

This paper makes the following key contributions:

- Developed a scalable sentiment analysis pipeline tailored for YouTube comments using BERT and traditional baselines.
- Incorporated class balancing techniques (SMOTE, class weights, backtranslation) and benchmarked their impact.
- Integrated SHAP, LIME, and Captum for explainability in both evaluation and deployment phases.
- Designed a real-time Streamlit dashboard with moderation and audit logging for safe, production-grade use.

X. CONCLUSION AND FUTURE WORK

In this research, I presented a fully operational sentiment analysis system for YouTube comments, combining transformer-based modeling, advanced preprocessing, explainability, and moderation. The system surpassed traditional machine learning benchmarks and addressed real-world deployment concerns like bias, toxicity, and interpretability.

Future improvements include:

- **Real-Time Streaming:** Integration with real-time comment streams via YouTube WebSockets or Pub/Sub.
- **Multilingual Support:** Extending to handle non-English comments using multilingual BERT or XLM-R models.
- **Model Distillation:** Reducing model size using knowledge distillation for faster inference in low-resource environments.
- **Fine-Grained Sentiment:** Moving beyond 3-class classification to include emotions like joy, anger, sadness, etc.

This work contributes to the growing intersection of ethical AI, explainability, and scalable NLP systems for social platforms.

The deployed application is publicly accessible via a Streamlit Cloud URL and has been showcased in the author’s resume and LinkedIn portfolio to demonstrate real-world, interpretable data science capabilities.

XI. ETHICAL CONSIDERATIONS

This work was developed with a strong focus on ethical AI deployment. All comments used were publicly available and collected via the official YouTube Data API, ensuring compliance with content access policies. Additionally, a rule-based override system was implemented to flag and moderate offensive or toxic content using curated keyword lists and regular expressions. This promotes responsible NLP practices and ensures that the deployed model avoids reinforcing or amplifying harmful language in any public or enterprise setting. No personally identifiable information (PII) was collected or stored.

ACKNOWLEDGMENT

This research project was undertaken as an individual effort by the author. All stages — from problem framing and model development to deployment and documentation — were carried out independently with the objective of building a production-grade, interpretable, and ethical AI system.

The author would like to acknowledge the open-source contributions of the Hugging Face Transformers library, Streamlit, and Captum, which made real-time deployment and explainability integration feasible. The guidance offered by community resources and documentation played a significant role in shaping this solution.

This paper stands as a testament to the power of self-directed learning, experimentation, and determination in bringing an idea from concept to reality.

REFERENCES

- [1] J. Wei and K. Zou, “Eda: Easy data augmentation techniques for boosting performance on text classification tasks,” in *Proceedings of EMNLP*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.11196>
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [3] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” pp. 1135–1144, 2016.
- [4] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] Google Cloud, “Youtube data api v3,” <https://developers.google.com/youtube/v3>, accessed: 2024-04-07.
- [6] Streamlit Inc., “Streamlit: The fastest way to build and share data apps,” <https://streamlit.io>, accessed: 2024-04-07.
- [7] A. Bibal and B. Frénay, “Combining machine learning and rule-based systems for content moderation,” *Journal of Artificial Intelligence Research*, vol. 53, pp. 399–434, 2015.
- [8] Hugging Face, “Hugging face transformers,” <https://huggingface.co/transformers>, accessed: 2024-04-07.